SCV SECURITY

AUDIT REPORT

BlockX

BlockX-Spin

Prepared by SCV-Security

On 21st March 2025

Table of Contents

Table of Contents **2**
Introduction **3**
Scope Functionality 3
Submitted Codebase 3
Revisions – Submitted Codebase 3
Methodologies 4
Security Assessment Questionnaire 6
Security Assessment Results 7
Code Criteria 9
Findings Summary **10**
Findings Technical Details **11**
1. Attackers can cause user funds to be stuck with zero bet amount 11
2. Malicious chains can manipulate the roulette result to steal funds 12
3. Denial of service due to unbounded iteration 14
4. MINIMUMRESERVE does not account for different denoms 16
5. Rand generator contract configuration cannot be updated 17
6. Admin cannot halt the contract 18
7. Incorrect comparison during validate_maximum_reward_exceed 19
8. Missing validation when adding rooms 20
9. Misleading query response returned 22
10. Addresses are not validated 23
11. Contract name and version are not updated after migration 24
12. execute_change_room_config does not check the NFT's validity 25
13. Two-step ownership transfer is not implemented 26
14. Ineffective execute_deposit function 27
Document Control **28**
Appendices **29**
A. Appendix - Risk assessment methodology 29
B. Appendix - Report Disclaimer 30

Introduction

SCV has been engaged by NFTSpin to conduct a comprehensive security review with the goal of identifying potential security threats and vulnerabilities within the codebase. The purpose of this audit is to evaluate the security posture of the codebase and provide actionable recommendations to mitigate any identified risks. This report presents an overview of the findings from our security audit, outlining areas of concern and proposing effective measures to enhance the codebase's security.

Scope Functionality

The Rand Generator and Roulette-SC contract feature a fully decentralized gaming system. Users deposit funds into the Roulette-SC contract within a specified amount of time along with their chosen bet direction. The Rand Generator contract will determine the result, where the random number is queried from BandChain using IBC.

Submitted Codebase

Roulette Contracts	
Repository	https://github.com/NFTSpin/Roulette-SC
Commit	`fd72ef2a378e4a81129de3f5fcbe03fc9acdece3`
Branch	`main`

Rand Generator	
Repository	https://github.com/NFTSpin/RandGenerator
Commit	`88d2ed43ee8ce433e3ea3beaf2b5274f4ff5c1c7`
Branch	`main`

Revisions – Submitted Codebase

Roulette Contracts	
Repository	https://github.com/NFTSpin/Roulette-SC
Commit	`f4a66748e58eb84674b9c83fb65b69332bc02e10`
Branch	`main`

Rand Generator	
Repository	https://github.com/NFTSpin/RandGenerator
Commit	`5959f256cd2c310c2a1351dd30394af9656d22fa`
Branch	`main`

Methodologies

SCV performs a combination of automated and manual security testing based on the scope of testing. The testing performed is based on the extensive experience and knowledge of the auditor to provide the greatest coverage and value to NFTSpin. Testing includes, but is not limited to, the following:

- Understanding the application and its functionality purpose.
- Deploying SCV in-house tooling to automate dependency analysis and static code review.
- Analyse each line of the code base and inspect application security perimeter.
- Review underlying infrastructure technologies and supply chain security posture.

Security Assessment Questionnaire

In addition to the above methodologies, SCV conducted a targeted (Xion's app-chain context) questionnaire-based assessment to address key security concerns, ensuring a thorough evaluation of potential risks. The following aspects were specifically examined:

1. **Administrator Privileges & Fund Security**
   - 1.1. Do contract administrators or owners have privileged functions that could negatively impact users, such as the ability to steal or block funds?
   - 1.2. Are administrative controls appropriately restricted to minimize security risks while maintaining necessary functionality?

2. **Mitigation of Fee Drain via Spam Vectors**
   - 2.1. Could certain contract functions be exploited to drain fee funds, particularly within the Xion fee-sponsoring model?
   - 2.2. Do execution functions include self-limiting mechanisms, such as cost implications or configurable constraints, to prevent spam-based depletion of resources?

Security Assessment Results

1. Administrator Privileges & Fund Security

- Rand generator contract:

  No unauthorized fund movement or blocking mechanisms were identified. The only available entry point is the `Request` message, which can only be called by the `OPERATOR`. We note that this contract is not expected to hold funds.

- Roulette SC contract:

  The contract does not provide administrators with direct control over user funds. Privileged functions are restricted to maintenance operations and are secured via role-based access controls.

  There is a `WithdrawFromPool` function that allows the room owner to withdraw excess funds from the contract when compared with the `MINIMUMRESERVE` state, however, it should be restricted to the profit portion amount only.

2. **Mitigation of Fee Drain via Spam Vectors**

- Rand generator contract:

  No funds wastage or resource depletion attack vectors were identified. The available entry point (`Request` message) is a privileged function that only the `OPERATOR` can call.

- Roulette SC contract:

  The contract includes built-in rate-limiting measures and configurable constraints to prevent abuse of fee sponsorship. For example, the `Bet` message implements a `min_bet` restriction, preventing users from betting zero amount to abuse the fee grants.

  However, the `AddRoom` and `UpdateBetLimit` functions do not implement validations to ensure the `min_bet` amount is not set to zero.

  Additionally, the room owner (who is nominated by the admin) could potentially spam the `WithdrawFromPool` and `UpdateBetLimit` messages to withdraw zero amounts of CW20 tokens or perform zero change updates.

  If this occurs, the admin can call the `ChangeRoomConfig` message to update the room owner's address to restrict further spam transactions. Alternatively, consider returning an error if the `amount` to withdraw is zero or the new `max_bet` and `min_bet` limits are the same as the current one.

Code Criteria

This section provides an evaluation of specific criteria aspects as described below:

- **Documentation:** Evaluating the presence and comprehensiveness of publicly available or provided explanatory information, diagram flowcharts, comments, and supporting documents to enhance code understanding.
- **Coverage:** Evaluating whether the code adequately addresses all necessary cases and scenarios, ensuring that the intended functionality or requirements are sufficiently covered.
- **Readability:** Assessing how easily the code can be understood and maintained, considering factors such as code structure, naming conventions, and overall organisation.
- **Complexity:** Evaluating the complexity of the code, including factors such as, number of lines, conditional statements, and nested structures.

The status of each criteria is categorised as either **SUFFICIENT** or **NOT-SUFFICIENT** based on the audit assessment. This categorisation provides insights to identify areas that may require further attention and improvement.

Criteria	Status	Notes
Documentation	**NOT-SUFFICIENT**	There is no documentation available.
Coverage	**NOT-SUFFICIENT**	Coverage can be improved as `cargo tarpaulin` reports the following: • Rand Generator: `9.09%` • Roulette-SC: `64.93%`
Readability	**NOT-SUFFICIENT**	A few issues were found that directly affected the code readability: 1) Deprecated and unused functions are found in the codebase. 2) Run "`cargo clippy`" to increase code readability and maintainability.
Complexity	**SUFFICIENT**	N/A

Findings Summary

Summary Title	Risk Impact	Status
Attackers can cause user funds to be stuck with zero bet amount	CRITICAL	RESOLVED
Malicious chains can manipulate the roulette result to steal funds	CRITICAL	RESOLVED
Denial of service due to unbounded iteration	CRITICAL	RESOLVED
`MINIMUMRESERVE` does not account for different denoms	SEVERE	RESOLVED
Rand generator contract configuration cannot be updated	MODERATE	RESOLVED
Admin cannot halt the contract	MODERATE	RESOLVED
Incorrect comparison during `validate_maximum_reward_exceed`	MODERATE	RESOLVED
Missing validation when adding rooms	LOW	RESOLVED
Misleading query response returned	LOW	RESOLVED
Addresses are not validated	LOW	RESOLVED
Contract name and version are not updated after migration	LOW	RESOLVED
`execute_change_room_config` does not check the NFT's validity	LOW	RESOLVED
Two-step ownership transfer is not implemented	INFO	RESOLVED
Ineffective `execute_deposit` function	INFO	RESOLVED

Findings Technical Details

1. Attackers can cause user funds to be stuck with zero bet amount

RISK IMPACT: CRITICAL	STATUS: RESOLVED

Description

In `src/execute.rs:330-343`, the `distribute_reward_to_users` function computes the reward amount and distributes it to users. This is computed from the user's bet amount (`bet.amount`), which is configured when the user creates their bet in the `execute_bet` function.

The issue is that the `execute_bet` function does not return an error if any bet amounts are zero. If the bet amount is zero, the `distribute_reward_to_users` function will compute the `reward_without_fee` variable as zero in `src/execute.rs:330`, causing the contract to attempt to distribute zero native funds to the user.

As a result, the `execute_close` function will fail because Cosmos SDK does not allow zero native funds to be transferred, causing a denial of service.

Recommendation

Consider returning an error if the `bet.amount` is zero in the `execute_bet` function.

2. Malicious chains can manipulate the roulette result to steal funds

RISK IMPACT: CRITICAL	STATUS: RESOLVED

Description

Upon establishing a successful handshake, the `ENDPOINT` state records the channel information in `contracts/rand-generator/src/ibc.rs:45`. This state is used when requesting an `OracleRequestPacketData` in `contracts/rand-generator/src/contract.rs:74`. The BandChain will send a packet containing a random number to `contracts/rand-generator/src/ibc.rs:88`, eventually calling the `RouletteExecuteMsg::CloseRound` to the roulette address with the `winner` number.

The issue is that malicious chains can connect with the rand generator contract, overwriting the `ENDPOINT` state. When the operator requests an `OracleRequestPacketData`, the request is sent to the malicious chain instead of BandChain. The malicious chain can then respond with a chosen `rand_number` (see `contracts/rand-generator/src/ibc.rs:111`) to manipulate the roulette result, giving them an unfair advantage and winning user funds.

Recommendation

Consider applying the following recommendations:

1. In the `ibc_channel_connect` function, remove the `ENDPOINT` state and emit only the connected channel ID value (see example). After connecting with BandChain, the channel ID can be retrieved from the transaction hash.
2. Implement a privileged function that allows the operator to configure the BandChain channel ID and record it in the `ENDPOINT` state (see example). This ensures the `ENDPOINT` state recorded the correct channel ID.
3. In the `ibc_packet_receive` function, implement a validation that returns an error if the packet's destination channel ID does not match the recorded

channel ID from the `ENDPOINT` state (see example). This ensures that malicious chains cannot send crafted packets to game the roulette.

3. Denial of service due to unbounded iteration

RISK IMPACT: CRITICAL	STATUS: RESOLVED

Description

Across the Roulette-SC codebase, there are a few instances where iterations are unbounded:

- In `src/execute.rs:306`, the `distribute_reward_to_users` function iterates over all the rooms created, retrieves all the user bets, and computes the reward and fee amount. This iteration is unbounded and will fail if there are too many rooms, players, or bets.

  Attackers can purposely cause this function to fail by betting with many different addresses and bet types. As a result, the `CloseRound` message will always fail due to an out-of-gas error, preventing users from receiving their rewards and locking user funds.

- In `src/execute.rs:633`, the `validate_maximum_reward_exceed` function iterates over all direction points for the room ID in this round. This iteration is unbounded and will fail if there are too many players and bets.

  Similar to the above, attackers can purposely cause this function to fail by betting with many different addresses and bet types. As a result, the `Bet` message will always fail due to an out-of-gas error.

- In `src/execute.rs:795`, the `query_all_members_one_round_room` function performs an unbounded iteration to retrieve all the players for this room ID and round ID. The function will fail if too many players are registered due to an out-of-gas error.

Recommendation

Consider limiting the iteration capacity to be within a maximum limit. This can be achieved by the following:

- Implement a maximum number of players who can join a specific round or room. This requires performing extensive testing to determine the appropriate limit.
- Implement a maximum number of bets a player can create.
- Reduce the computations required during iterations. For example, the `point_ratio_info` variable in `src/execute.rs:666` can be stored in the `BetInfo` to avoid calling the `get_points_ratio_information` every time.
- Implement a mechanism to remove or deprecate rooms from user bets if not all rooms are required for betting.
- Implement a pagination mechanism to process iterations in batches instead of all entries at once with a state machine approach. This allows the required iterations to be offloaded and spread across multiple transactions.

4. MINIMUMRESERVE does not account for different denoms

RISK IMPACT: SEVERE	STATUS: RESOLVED

Description

The `MINIMUMRESERVE` state is responsible for enforcing the minimum reserve amount of funds available in the contract to process roulette rewards correctly.

However, the state simply records the token amount without considering the token denom. This is problematic because if rooms have different token denoms configured, the `MINIMUMRESERVE` state will incorrectly sum them together, as seen in `src/execute.rs:194-203`.

This makes the `MINIMUMRESERVE` state unreliable, which has severe consequences for the contract. For example, the validation in `src/execute.rs:689-698` will be ineffective, and the withdrawal amount computed in `src/execute.rs:618-620` will be incorrect.

Recommendation

Consider modifying the `MINIMUMRESERVE` state to distinguish between the token denoms in the contract. This can be achieved by changing the `MINIMUMRESERVE` state to be a `Map`. The mapping key will represent the room's token denom, and the mapping value will record the balance.

This will allow the `validate_maximum_reward_exceed` function to correctly access the balance amount for the particular room's token denom. The caveat is that all rooms must be configured with different denoms to avoid incorrect computation, which is detailed in the "Missing validation when adding rooms" issue.

5. Rand generator contract configuration cannot be updated

RISK IMPACT: MODERATE	STATUS: RESOLVED

Description

The `try_request` function in `contracts/rand-generator/src/contract.rs:73` requests the latest Oracle price from BandChain using the configurations in the `BAND_CONFIG` state.

However, since there is no entry point to update the `BAND_CONFIG` state, the operator has no control over the `OracleRequestPacketData` parameters when sending the packet. For example, if the request fails due to an insufficient fee limit (see `contracts/rand-generator/src/contract.rs:107`), the operator cannot update the `Config.fee_limit` field to retry the failed request.

Recommendation

Consider implementing a privileged entry point for the operator to update the `BAND_CONFIG` state.

6. Admin cannot halt the contract

RISK IMPACT: MODERATE	STATUS: RESOLVED

Description

The `State.is_haulted` field in `src/state.rs:28` determines whether the contract has been halted. If it is halted, users will be unable to bet due to the `assert_not_haulted` validation in `src/execute.rs:186`.

However, there is no entry point implemented for the admin to update the `State.is_haulted` field in the contract. This prevents the admin from pausing the contract in case an exploit occurs.

Recommendation

Consider implementing a privileged entry point for the admin to update the `State.is_haulted` field.

7. Incorrect comparison during `validate_maximum_reward_exceed`

RISK IMPACT: MODERATE	STATUS: RESOLVED

Description

In `src/execute.rs:643-648`, the `validate_maximum_reward_exceed` function queries the token balance in the contract. This is called when users add new bets to the contract.

However, if the room's token denom is a native denom, the `validate_maximum_reward_exceed` function does not decrease the contract balance by the native token deposit amount. This is required because when the `execute_bet` function is called, the contract balance has already increased with the user's deposit amount, which is enforced by the `validate_input_amount` validation in `src/execute.rs:192`.

As a result, the validation in `src/execute.rs:689-698` will be ineffective, potentially causing an insufficient funds error when distributing rewards.

Recommendation

Consider decreasing the contract's balance by the user's deposit amount if the room's token denom is a native denom.

8. Missing validation when adding rooms

RISK IMPACT: LOW	STATUS: RESOLVED

Description

In `src/execute.rs:103`, the `execute_add_room` function allows the admin to add `ROOMS`, which allows users to bet in it.

However, there are no validations implemented. This is problematic due to the following reasons:

- The `game_denom` configured could be similar to other rooms, causing incorrect computation when the `get_withdrawal_amount` function is called in `src/execute.rs:605`. As a result, the room owner can withdraw funds from other rooms.

  Additionally, there is a comment in `src/execute.rs:315` stating that bet denom is different across all rooms. However, it is not enforced.

- The `min_bet` and `max_bet` fields are not validated such that `min_bet` must be smaller than `max_bet`. This issue also occurs in the `execute_update_bet_limit` function (see `src/execute.rs:487-488`).

Recommendation

Consider applying the following recommendations:

- Ensure the `game_denom` configured across the rooms is unique. This can be achieved by storing the denom as a key in a map and validating whether the key exists when creating new rooms.

  This can be combined with the "`MINIMUMRESERVE` does not account for different denoms" issue by checking whether the token denom exists in the `MINIMUMRESERVE` map.

- Validate that `min_bet` is less than `max_bet` when creating rooms and updating bet limits. Additionally, consider returning an error if the `min_bet` is zero to prevent users from spamming transactions.

9. Misleading query response returned

RISK IMPACT: LOW	STATUS: RESOLVED

Description

In `src/query.rs:275`, the `query_winner_round` function returns the winner as `40` if the `WINNERNUMBER` for the round ID does not exist. This is problematic because if a winner's round ID is not found, it should return an error instead of returning a hardcoded value, which is misleading.

A similar issue also exists in `src/query.rs:258-260` where the `query_maximum_withdrwal` function returns the withdrawal amount as zero if the room ID does not exist.

Recommendation

Consider returning an error in the `query_winner_round` and `query_maximum_withdrwal` functions if the requested entry is not found.

10. Addresses are not validated

RISK IMPACT: LOW	STATUS: RESOLVED

Description

In a few instances of the codebase, the address fields are passed as `Addr`:

- Roulette address: `contracts/rand-generator/src/contract.rs:52` and `contracts/rand-generator/src/contract.rs:121`
- Operator address: `contracts/rand-generator/src/contract.rs:53` and `contracts/rand-generator/src/contract.rs:122`
- Admin address: `src/state.rs:18`
- NFT contract address: `src/state.rs:19`
- Distributor address: `src/state.rs:21`
- `rand_generator` contract address: `src/msg.rs:9` and `src/execute.rs:55`

This is problematic because these addresses are not checked to be valid with the `deps.api.addr_validate` function. Invalid addresses may occur when constructed with the `Addr::unchecked` function.

Recommendation

Consider validating the addresses with the `deps.api.addr_validate` function in the contract instantiation and update config phases.

11. Contract name and version are not updated after migration

RISK IMPACT: LOW	STATUS: RESOLVED

Description

In `contracts/rand-generator/src/contract.rs:123`, the `migrate` entry point does not call `set_contract_version` after a successful contract migration. This is problematic because the migrated contract still reflects the old contract information, potentially causing confusion and future migration failures.

This issue also affects the `migrate` entry point of the Roulette-SC contract, as seen in `src/execute.rs:46`.

Recommendation

Consider calling the `set_contract_version` function in the `migrate` entry point of the Rand Generator and Roulette-SC contracts.

12. `execute_change_room_config` does not check the NFT's validity

RISK IMPACT: LOW	STATUS: RESOLVED

Description

The `execute_change_room_config` function in `src/execute.rs:460` does not validate that the newly updated `nft_id` is valid by querying the NFT contract. As a result, invalid NFT IDs will not have an owner, leaving rewards earned in the contract and preventing the update of the betting limits.

Recommendation

Consider querying the NFT contract with the updated NFT ID to ensure the NFT is valid. For example, see `src/execute.rs:117-124`.

13. Two-step ownership transfer is not implemented

RISK IMPACT: INFORMATIONAL	STATUS: RESOLVED

Description

The codebase does not implement a two-step ownership transfer in `src/execute.rs:98` when updating the `admin` address.

Using a two-step ownership transfer mechanism helps provide a window of opportunity for the current owner to cancel the transfer if they did not intend to initiate it or if there were any unintended actions.

As a result, ownership will be lost and cannot be recovered if it is transferred to an incorrect address that no one owns.

Recommendation

Consider implementing a two-step ownership transfer functionality that proposes a new owner in the first step and requires the proposed owner to accept it as the second step.

14. Ineffective `execute_deposit` function

RISK IMPACT: INFORMATIONAL	STATUS: RESOLVED

Description

The `execute_deposit` function in `src/execute.rs:407` ensures that only the room owner can deposit funds into the contract. However, this is ineffective because anyone can perform a `Cw20ExecuteMsg::Transfer` or `BankMsg::Send` message to deposit funds directly to the contract.

Recommendation

Consider removing the `execute_deposit` function.

Document Control

Version	Date	Notes
-	28th February 2025	Security audit commencement date.
0.1	10th March 2025	Initial report with identified findings delivered.
0.5	20th March 2025	Fixes remediations implemented and reviewed.
1.0	21st March 2025	Audit completed, final report delivered.

Appendices

A. Appendix – Risk assessment methodology

SCV-Security employs a risk assessment methodology to evaluate vulnerabilities and identified issues. This approach involves the analysis of both the LIKELIHOOD of a security incident occurring and the potential IMPACT if such an incident were to happen. For each vulnerability, SCV-Security calculates a risk level on a scale of 5 to 1, where 5 denotes the highest likelihood or impact. Consequently, an overall risk level is derived from combining these two factors, resulting in a value from 10 to 1, with 10 signifying the most elevated level of security risk

Risk Level	Range
CRITICAL	10
SEVERE	From 9 to 8
MODERATE	From 7 to 6
LOW	From 5 to 4
INFORMATIONAL	From 3 to 1

LIKELIHOOD and **IMPACT** would be individually assessed based on the below:

Rate	LIKELIHOOD	IMPACT
5	**Extremely Likely**	Could result in severe and irreparable consequences.
4	**Likely**	May lead to substantial impact or loss.
3	**Possible**	Could cause partial impact or loss on a wide scale.
2	**Unlikely**	Might cause temporary disruptions or losses.
1	**Rare**	Could have minimal or negligible impact.

B. Appendix – Report Disclaimer

This report should not be regarded as an "endorsement" or "disapproval" of any specific project or team. These reports do not indicate the economics or value of any "product" or "asset" created by a team or project that engages SCV-Security for a security review. The audit report does not make any statements or warranties about the code's utility, safety, suitability of the business model, regulatory compliance of the business model, or any other claims regarding the fitness of the implementation for its purpose or its bug-free status. The audit documentation is intended for discussion purposes only. The content of this audit report is provided "as is," without representations and warranties of any kind, and SCV-Security disclaims any liability for damages arising from or in connection with this audit report. Copyright of this report remains with SCV-Security.

THANK YOU FOR CHOOSING



SCV SECURITY

scv.services

contact@scv.services